

```

{
    Int j,n1=x1.size(),n2=x2.size();
    Doub d1,d2,dum,dumm,fa,fb,fc,fd,ga,gb,gc,gd,r1,r2,rr,sqen;
    KSdist ks;
    d1=0.0;
    for (j=0;j<n1;j++) {
        First, use points in the first sample as origins.
        quadct(x1[j],y1[j],x1,y1,fa,fb,fc,fd);
        quadct(x1[j],y1[j],x2,y2,ga,gb,gc,gd);
        if (fa > ga) fa += 1.0/n1;
        if (fb > gb) fb += 1.0/n1;
        if (fc > gc) fc += 1.0/n1;
        if (fd > gd) fd += 1.0/n1;
        d1=MAX(d1,abs(fa-ga));
        d1=MAX(d1,abs(fb-gb));
        d1=MAX(d1,abs(fc-gc));
        d1=MAX(d1,abs(fd-gd));
    }
    d2=0.0;
    for (j=0;j<n2;j++) {
        Then, use points in the second sample as
        origins.
        quadct(x2[j],y2[j],x1,y1,fa,fb,fc,fd);
        quadct(x2[j],y2[j],x2,y2,ga,gb,gc,gd);
        if (ga > fa) ga += 1.0/n1;
        if (gb > fb) gb += 1.0/n1;
        if (gc > fc) gc += 1.0/n1;
        if (gd > fd) gd += 1.0/n1;
        d2=MAX(d2,abs(fa-ga));
        d2=MAX(d2,abs(fb-gb));
        d2=MAX(d2,abs(fc-gc));
        d2=MAX(d2,abs(fd-gd));
    }
    d=0.5*(d1+d2);
    Average the K-S statistics.
    sqen=sqrt(n1*n2/Doub(n1+n2));
    pearsn(x1,y1,r1,dum,dumm);
    pearsn(x2,y2,r2,dum,dumm);
    Get the linear correlation coefficient for each
    sample.
    rr=sqrt(1.0-0.5*(r1*r1+r2*r2));
    Estimate the probability using the K-S probability function.
    prob=ks.qks(d*sqen/(1.0+rr*(0.25-0.75/sqen)));
}

```

CITED REFERENCES AND FURTHER READING:

- Fasano, G. and Franceschini, A. 1987, "A Multidimensional Version of the Kolmogorov-Smirnov Test," *Monthly Notices of the Royal Astronomical Society*, vol. 225, pp. 155–170.[1]
- Peacock, J.A. 1983, "Two-Dimensional Goodness-of-Fit Testing in Astronomy," *Monthly Notices of the Royal Astronomical Society*, vol. 202, pp. 615–627.[2]
- Spergel, D.N., Piran, T., Loeb, A., Goodman, J., and Bahcall, J.N. 1987, "A Simple Model for Neutrino Cooling of the LMC Supernova," *Science*, vol. 237, pp. 1471–1473.[3]

14.9 Savitzky-Golay Smoothing Filters

In §13.5 we learned something about the construction and application of digital filters, but little guidance was given on *which particular* filter to use. That, of course, depends on what you want to accomplish by filtering. One obvious use for *low-pass* filters is to smooth noisy data.

The premise of data smoothing is that one is measuring a variable that is both slowly varying and also corrupted by random noise. Then it can sometimes be useful

to replace each data point by some kind of local average of surrounding data points. Since nearby points measure very nearly the same underlying value, averaging can reduce the level of noise without (much) biasing the value obtained.

We must comment editorially that the smoothing of data lies in a murky area, beyond the fringe of some better-posed, and therefore more highly recommended, techniques that are discussed elsewhere in this book. If you are fitting data to a parametric model, for example (see Chapter 15), it is almost always better to use raw data than to use data that have been pre-processed by a smoothing procedure. Another alternative to blind smoothing is so-called “optimal” or Wiener filtering, as discussed in §13.3 and more generally in §13.6. Data smoothing is probably most justified when it is used simply as a graphical technique, to guide the eye through a forest of data points all with large error bars, or as a means of making initial *rough* estimates of simple parameters from a graph.

In this section we discuss a particular type of low-pass filter, well-adapted for data smoothing, and termed variously *Savitzky-Golay* [1], *least-squares* [2], or *DISPO* (Digital Smoothing Polynomial) [3] filters. Rather than having their properties defined in the Fourier domain and then translated to the time domain, Savitzky-Golay filters derive directly from a particular formulation of the data smoothing problem in the time domain, as we will now see. Savitzky-Golay filters were initially (and are still often) used to render visible the relative widths and heights of spectral lines in noisy spectrometric data.

Recall that a digital filter is applied to a series of equally spaced data values $f_i \equiv f(t_i)$, where $t_i \equiv t_0 + i\Delta$ for some constant sample spacing Δ and $i = \dots - 2, -1, 0, 1, 2, \dots$. We have seen (§13.5) that the simplest type of digital filter (the nonrecursive or finite impulse response filter) replaces each data value f_i by a linear combination g_i of itself and some number of nearby neighbors,

$$g_i = \sum_{n=-n_L}^{n_R} c_n f_{i+n} \quad (14.9.1)$$

Here n_L is the number of points used “to the left” of a data point i , i.e., earlier than it, while n_R is the number used to the right, i.e., later. A so-called *causal* filter would have $n_R = 0$.

As a starting point for understanding Savitzky-Golay filters, consider the simplest possible averaging procedure: For some fixed $n_L = n_R$, compute each g_i as the average of the data points from f_{i-n_L} to f_{i+n_R} . This is sometimes called *moving window averaging* and corresponds to equation (14.9.1) with constant $c_n = 1/(n_L + n_R + 1)$. If the underlying function is constant, or is changing linearly with time (increasing or decreasing), then no bias is introduced into the result. Higher points at one end of the averaging interval are on the average balanced by lower points at the other end. A bias is introduced, however, if the underlying function has a nonzero second derivative. At a local maximum, for example, moving window averaging always reduces the function value. In the spectrometric application, a narrow spectral line has its height reduced and its width increased. Since these parameters are themselves of physical interest, the bias introduced is distinctly undesirable.

Note, however, that moving window averaging does preserve the area under a spectral line, which is its zeroth moment, and also (if the window is symmetric with $n_L = n_R$) its mean position in time, which is its first moment. What is violated is

the second moment, equivalent to the line width.

The idea of Savitzky-Golay filtering is to find filter coefficients c_n that preserve higher moments. Equivalently, the idea is to approximate the underlying function within the moving window not by a constant (whose estimate is the average), but by a polynomial of higher order, typically quadratic or quartic: For each point f_i , we least-squares fit a polynomial to all $n_L + n_R + 1$ points in the moving window, and then set g_i to be the value of that polynomial at position i . (If you are not familiar with least-squares fitting, you might want to look ahead to Chapter 15.) We make no use of the value of the polynomial at any other point. When we move on to the next point f_{i+1} , we do a whole new least-squares fit using a shifted window.

All these least-squares fits would be laborious if done as described. Luckily, since the process of least-squares fitting involves only a linear matrix inversion, the coefficients of a fitted polynomial are themselves linear in the values of the data. That means that we can do all the fitting in advance, for fictitious data consisting of all zeros except for a single 1, and then do the fits on the real data just by taking linear combinations. This is the key point, then: There are particular sets of filter coefficients c_n for which equation (14.9.1) “automatically” accomplishes the process of polynomial least-squares fitting inside a moving window.

To derive such coefficients, consider how g_0 might be obtained: We want to fit a polynomial of degree M in i , namely $a_0 + a_1 i + \dots + a_M i^M$, to the values $f_{-n_L}, \dots, f_{n_R}$. Then g_0 will be the value of that polynomial at $i = 0$, namely a_0 . The design matrix for this problem (§15.4) is

$$A_{ij} = i^j \quad i = -n_L, \dots, n_R, \quad j = 0, \dots, M \quad (14.9.2)$$

and the normal equations for the vector of a_j 's in terms of the vector of f_i 's is in matrix notation

$$(\mathbf{A}^T \cdot \mathbf{A}) \cdot \mathbf{a} = \mathbf{A}^T \cdot \mathbf{f} \quad \text{or} \quad \mathbf{a} = (\mathbf{A}^T \cdot \mathbf{A})^{-1} \cdot (\mathbf{A}^T \cdot \mathbf{f}) \quad (14.9.3)$$

We also have the specific forms

$$\{\mathbf{A}^T \cdot \mathbf{A}\}_{ij} = \sum_{k=-n_L}^{n_R} A_{ki} A_{kj} = \sum_{k=-n_L}^{n_R} k^{i+j} \quad (14.9.4)$$

and

$$\{\mathbf{A}^T \cdot \mathbf{f}\}_j = \sum_{k=-n_L}^{n_R} A_{kj} f_k = \sum_{k=-n_L}^{n_R} k^j f_k \quad (14.9.5)$$

Since the coefficient c_n is the component a_0 when $\mathbf{f}$ is replaced by the unit vector $\mathbf{e}_n$, $-n_L \leq n < n_R$, we have

$$c_n = \{(\mathbf{A}^T \cdot \mathbf{A})^{-1} \cdot (\mathbf{A}^T \cdot \mathbf{e}_n)\}_0 = \sum_{m=0}^M \{(\mathbf{A}^T \cdot \mathbf{A})^{-1}\}_{0m} n^m \quad (14.9.6)$$

Equation (14.9.6) says that we need only one row of the inverse matrix. (Numerically we can get this by LU decomposition with only a single backsubstitution.)

The function `savgol`, below, implements equation (14.9.6). As input, it takes the parameters `nL` = n_L , `nr` = n_R , and `m` = M (the desired order). Also input

M	n_L	n_R	Sample Savitzky-Golay Coefficients										
2	2	2	-0.086 0.343 0.486 0.343 -0.086										
2	3	1	-0.143 0.171 0.343 0.371 0.257										
2	4	0	0.086 -0.143 -0.086 0.257 0.886										
2	5	5	-0.084	0.021	0.103	0.161	0.196	0.207	0.196	0.161	0.103	0.021	-0.084
4	4	4	0.035 -0.128 0.070 0.315 0.417 0.315 0.070 -0.128 0.035										
4	5	5	0.042	-0.105	-0.023	0.140	0.280	0.333	0.280	0.140	-0.023	-0.105	0.042

is np , the physical length of the output array c , and a parameter ld that for data fitting should be zero. In fact, ld specifies which coefficient among the a_i 's should be returned, and we are here interested in a_0 . For another purpose, namely the computation of numerical derivatives (already mentioned in §5.7), the useful choice is $ld \geq 1$. With $ld = 1$, for example, the filtered first derivative is the convolution (14.9.1) divided by the stepsize Δ . For $ld = k > 1$, the array c must be multiplied by $k!$ to give derivative coefficients. For derivatives, one usually wants $m = 4$ or larger.

```
void savgol(VecDoub_O &c, const Int np, const Int nl, const Int nr,
           const Int ld, const Int m)
```

savgol.h

Returns in $c[0..np-1]$, in wraparound order (N.B.!) consistent with the argument `respsns` in routine `convlv`, a set of Savitzky-Golay filter coefficients. nl is the number of leftward (past) data points used, while nr is the number of rightward (future) data points, making the total number of data points used $nl + nr + 1$. ld is the order of the derivative desired (e.g., $ld = 0$ for smoothed function. For the derivative of order k , you must multiply the array c by $k!$.) m is the order of the smoothing polynomial, also equal to the highest conserved moment; usual values are $m = 2$ or $m = 4$.

```
{
    Int j,k,imj,ipj,kk,mm;
    Doub fac,sum;
    if (np < nl+nr+1 || nl < 0 || nr < 0 || ld > m || nl+nr < m)
        throw("bad args in savgol");
    VecInt indx(m+1);
    MatDoub a(m+1,m+1);
    VecDoub b(m+1);
    for (ipj=0;ipj<=(m << 1);ipj++) {          Set up the normal equations of the desired
        sum=(ipj ? 0.0 : 1.0);                  least-squares fit.
        for (k=1;k<=nr;k++) sum += pow(Doub(k),Doub(ipj));
        for (k=1;k<=nl;k++) sum += pow(Doub(-k),Doub(ipj));
        mm=MIN(ipj,2*m-ipj);
        for (imj = -mm;imj<=mm;imj+=2) a[(ipj+imj)/2][(ipj-imj)/2]=sum;
    }
    LUdcmp alud(a);                             Solve them: LU decomposition.
    for (j=0;j<m+1;j++) b[j]=0.0;
    b[ld]=1.0;
    Right-hand side vector is unit vector, depending on which derivative we want.
    alud.solve(b,b);                             Get one row of the inverse matrix.
    for (kk=0;kk<np;kk++) c[kk]=0.0;            Zero the output array (it may be bigger than
    for (k = -nl;k<=nr;k++) {                   number of coefficients).
        sum=b[0];                               Each Savitzky-Golay coefficient is the dot
        fac=1.0;                                product of powers of an integer with the
        for (mm=1;mm<=m;mm++) sum += b[mm]*(fac *= k);    inverse matrix row.
        kk=(np-k) % np;                         Store in wraparound order.
        c[kk]=sum;
    }
}
```

As output, `savgol` returns the coefficients c_n , for $-n_L \leq n \leq n_R$. These are stored in `c` in “wraparound order”; that is, c_0 is in `c[0]`, c_{-1} is in `c[1]`, and so on for further negative indices. The value c_1 is stored in `c[np-1]`, c_2 in `c[np-2]`, and so on for positive indices. This order may seem arcane, but it is the natural one where causal filters have nonzero coefficients in low array elements of `c`. It is also the order required by the function `convlv` in §13.1, which can be used to apply the digital filter to a data set.

The table on the previous page shows some typical output from `savgol`. For orders 2 and 4, the coefficients of Savitzky-Golay filters with several choices of n_L and n_R are shown. The central column is the coefficient applied to the data f_i in obtaining the smoothed g_i . Coefficients to the left are applied to earlier data, to the right, to later. The coefficients always add (within roundoff error) to unity. One sees that, as befits a smoothing operator, the coefficients always have a central positive lobe, but with smaller, outlying corrections of both positive and negative sign. In practice, the Savitzky-Golay filters are most useful for much larger values of n_L and n_R , since these few-point formulas can accomplish only a relatively small amount of smoothing.

Figure 14.9.1 shows a numerical experiment using a 33-point smoothing filter, that is, $n_L = n_R = 16$. The upper panel shows a test function, constructed to have six “bumps” of varying widths, all of height 8 units. To this function Gaussian white noise of unit variance has been added. (The test function without noise is shown as the dotted curves in the center and lower panels.) The widths of the bumps (full width at half of maximum, or FWHM) are 140, 43, 24, 17, 13, and 10, respectively.

The middle panel of Figure 14.9.1 shows the result of smoothing by a moving window average. One sees that the window of width 33 does quite a nice job of smoothing the broadest bump, but that the narrower bumps suffer considerable loss of height and increase of width. The underlying signal (dotted) is very badly represented.

The lower panel shows the result of smoothing with a Savitzky-Golay filter of the identical width and degree $M = 4$. One sees that the heights and widths of the bumps are quite extraordinarily preserved. A trade-off is that the broadest bump is less smoothed. That is because the central positive lobe of the Savitzky-Golay filter coefficients fills only a fraction of the full 33-point width. As a rough guideline, best results are obtained when the full width of the degree 4 Savitzky-Golay filter is between 1 and 2 times the FWHM of desired features in the data. (References [3] and [4] give additional practical hints.)

Figure 14.9.2 shows the result of smoothing the same noisy “data” with broader Savitzky-Golay filters of three different orders. Here we have $n_L = n_R = 32$ (65-point filter) and $M = 2, 4, 6$. One sees that, when the bumps are too narrow with respect to the filter size, even the Savitzky-Golay filter must at some point give out. The higher-order filter manages to track narrower features, but at the cost of less smoothing on broad features.

To summarize: Within limits, Savitzky-Golay filtering does manage to provide smoothing without loss of resolution. It does this by assuming that relatively distant data points have some significant redundancy that can be used to reduce the level of noise. The specific nature of the assumed redundancy is that the underlying function should be locally well-fitted by a polynomial. When this is true, as it is for smooth line profiles not too much narrower than the filter width, the performance of

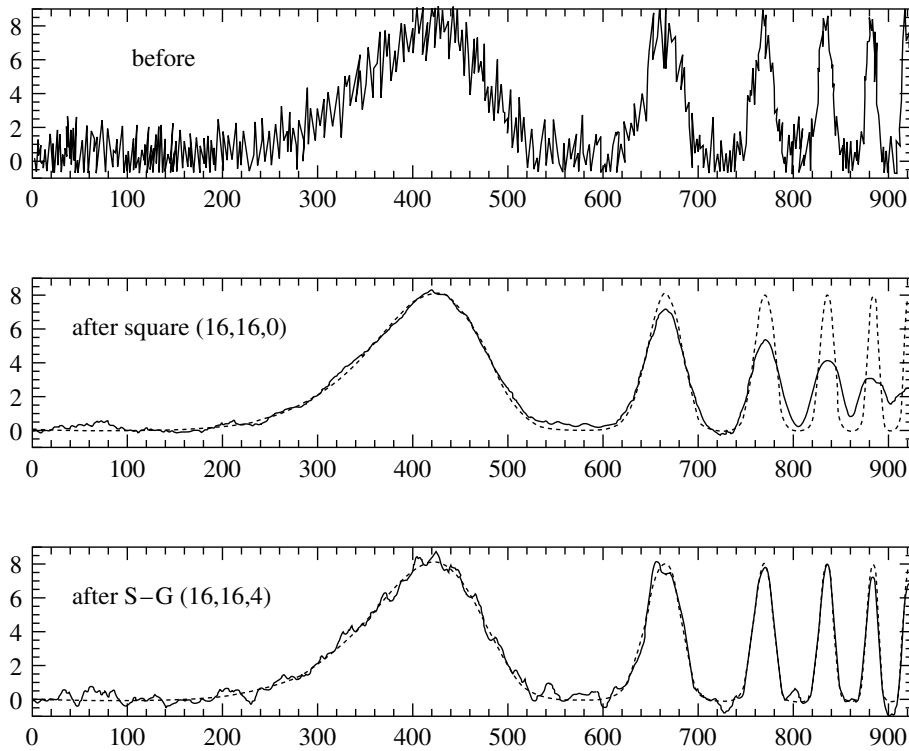


Figure 14.9.1. Top: Synthetic noisy data consisting of a sequence of progressively narrower bumps and additive Gaussian white noise. Center: Result of smoothing the data by a simple moving window average. The window extends 16 points leftward and rightward, for a total of 33 points. Note that narrow features are broadened and suffer corresponding loss of amplitude. The dotted curve is the underlying function used to generate the synthetic data. Bottom: Result of smoothing the data by a Savitzky-Golay smoothing filter (of degree 4) using the same 33 points. While there is less smoothing of the broadest feature, narrower features have their heights and widths preserved.

Savitzky-Golay filters can be spectacular. When it is not true, these filters have no compelling advantage over other classes of smoothing filter coefficients.

A last remark concerns irregularly sampled data, where the values f_i are not uniformly spaced in time. The obvious generalization of Savitzky-Golay filtering would be to do a least-squares fit within a moving window around each data point, one containing a fixed number of data points to the left (n_L) and right (n_R). Because of the irregular spacing, however, there is no way to obtain universal filter coefficients applicable to more than one data point. One must instead do the actual least-squares fits for each data point. This becomes computationally burdensome for larger n_L , n_R , and M .

As a cheap alternative, one can simply pretend that the data points *are* equally spaced. This amounts to virtually shifting, within each moving window, the data points to equally spaced positions. Such a shift introduces the equivalent of an additional source of noise into the function values. In those cases where smoothing is useful, this noise will often be much smaller than the noise already present. Specifically, if the location of the points is approximately random within the window, then a rough

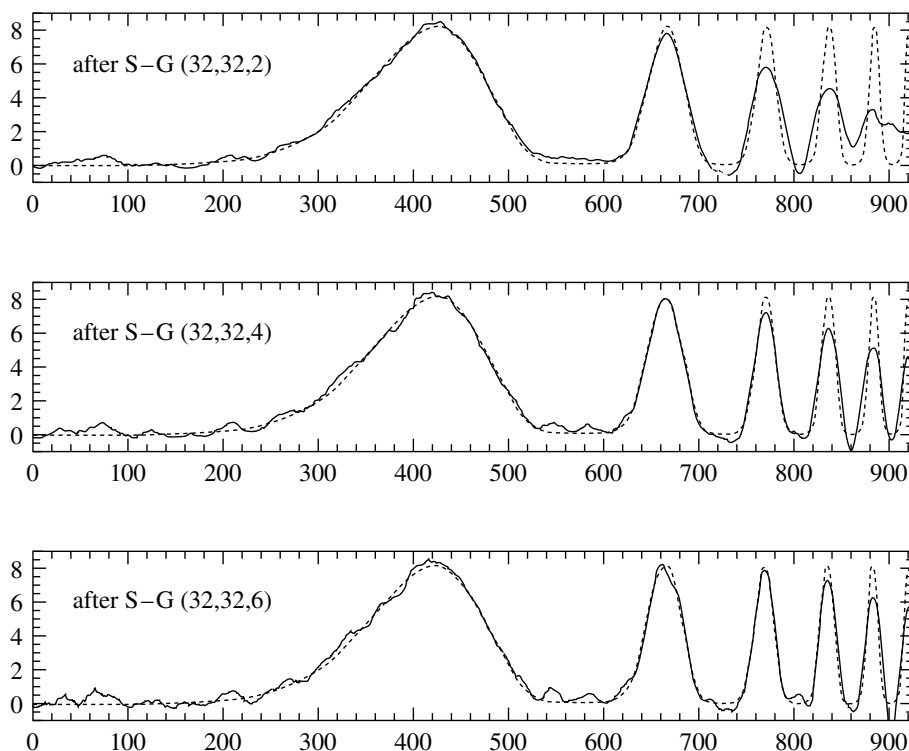


Figure 14.9.2. Result of applying wider 65-point Savitzky-Golay filters to the same data set as in Figure 14.9.1. Top: degree 2. Center: degree 4. Bottom: degree 6. All of these filters are inoptimally broad for the resolution of the narrow features. Higher-order filters do best at preserving feature heights and widths, but do less smoothing on broader features.

criterion is this: If the change in f across the full width of the $N = n_L + n_R + 1$ point window is less than $\sqrt{N/2}$ times the measurement noise on a single point, then the cheap method can be used.

CITED REFERENCES AND FURTHER READING:

- Savitzky A., and Golay, M.J.E. 1964, "Smoothing and Differentiation of Data by Simplified Least Squares Procedures," *Analytical Chemistry*, vol. 36, pp. 1627–1639.[1]
 Hamming, R.W. 1983, *Digital Filters*, 2nd ed. (Englewood Cliffs, NJ: Prentice-Hall).[2]
 Ziegler, H. 1981, "Properties of Digital Smoothing Polynomial (DISPO) Filters," *Applied Spectroscopy*, vol. 35, pp. 88–92.[3]
 Bromba, M.U.A., and Ziegler, H. 1981, "Application Hints for Savitzky-Golay Digital Smoothing Filters," *Analytical Chemistry*, vol. 53, pp. 1583–1586.[4]